



College of Engineering
Mechatronics and mechanical engineering department
MECA444-Mechatronics Systems Programming

Fall 2024

Capstone Project

Report Submitted by:

Ahmad Achaji – 20210313

Ahmad Hammoudeh – 20210482

Semester: Fall 2024

Report Submitted to:

Ahmad Koubeissi

Contents

1.Introduction.....	3
2. Overall Functionality	3
Temperature Control Subsystem:	3
Lighting Control Subsystem:	3
Security System Subsystem:.....	3
Central User Interface Node:	3
3. Detailed System Components	4
3.1 Nodes	4
3.2 Topics	4
3.3 Services	5
3.4 Actions	5
4. rqt_graph Analysis	5
5.System file and nodes	6
6. System Flow Summary.....	6
7. Conclusion	7

1. Introduction

The Smart Home Control System is designed to integrate key functionalities that simulate temperature control, lighting control, and security management. The system is implemented using the Robot Operating System 2 (ROS2) framework, leveraging topics, services, and actions to facilitate modular and interactive communication between nodes.

This project aims to achieve the following objectives:

- Simulate temperature monitoring and climate control using a thermostat system.
- Enable lighting control with manual ON/OFF functionality and gradual light dimming using an action server.
- Provide a security state monitoring system to simulate an armed, disarmed, or triggered alarm state.
- Allow users to interact with the system through a central user interface node.

2. Overall Functionality

The Smart Home Control System consists of three major subsystems:

Temperature Control Subsystem:

Simulates a temperature sensor that generates random temperature values.

Includes a climate control system (thermostat) that regulates the heater or cooler to maintain a desired temperature.

Allows users to specify the desired temperature via a ROS2 service `/set_temperature`.

Lighting Control Subsystem:

Provides basic lighting control (ON/OFF) and advanced dimming functionality.

Implements an action server `/dim_lights` for smooth and gradual light dimming.

Provides real-time feedback and status updates on the progress of dimming.

Security System Subsystem:

Simulates a security system with states: ARMED, DISARMED, or TRIGGERED.

Publishes the current state of the security system via the `/security_state` topic.

Central User Interface Node:

Serves as the main control hub for interacting with all subsystems.

Allows users to:

- Send temperature commands.
- Control lighting (ON/OFF or dimming).
- Arm or disarm the security system.
- Subscribes to feedback topics to provide users with system status updates.

3. Detailed System Components

3.1 Nodes

The system consists of the following ROS2 nodes:

1. `/user_interface_node`:
 - Acts as the main interface for user input.
 - Sends commands to the `/temperature_control_node` via the `/temperature_command` topic.
 - Controls the `/light_control_node` using the `/light_command` topic or the `/dim_lights` action server.
 - Monitors the state of the security system by subscribing to the `/security_state` topic.
2. `/temperature_control_node`:
 - Generates random temperature readings to simulate a temperature sensor.
 - Regulates a simulated thermostat (heater or cooler) to achieve a desired temperature.
 - Provides a ROS2 service `/set_temperature` to receive the desired temperature setpoint.
3. `/light_control_node`:
 - Controls lighting states (ON/OFF) based on commands received from the `/light_command` topic.
 - Implements the `/dim_lights` action server for gradual light dimming, providing:
 - Feedback on the progress of dimming.
 - Status updates on task completion or cancellation.
4. `/security_node`:
 - Monitors and manages the state of the security system (ARMED/DISARMED/TRIGGERED).
 - Publishes updates to the `/security_state` topic.

3.2 Topics

Topic	Publisher	Subscriber	Description
<code>/temperature_command</code>	<code>user_interface_node</code>	<code>temperature_control_node</code>	Used to send the desired temperature to the thermostat system.
<code>/light_command</code>	<code>user_interface_node</code>	<code>light_control_node</code>	Sends commands to turn lights ON, OFF, or adjust brightness.
<code>/security_state</code>	<code>security_node</code>	<code>user_interface_node</code>	Publishes the current security system state (e.g., ARMED, DISARMED, TRIGGERED).

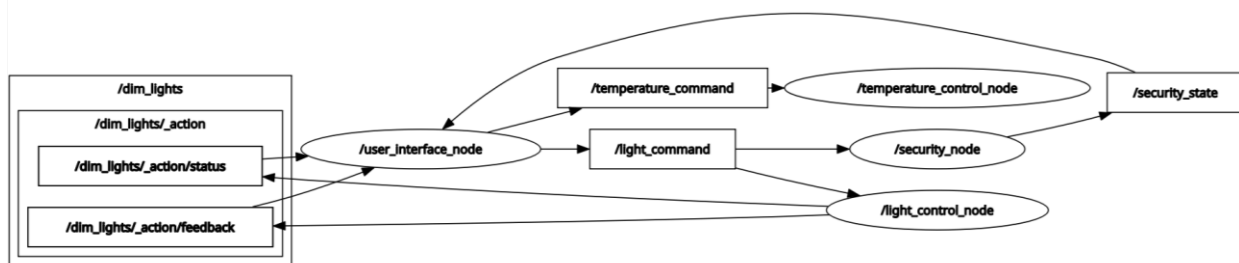
3.3 Services

Service	Node	Purpose
/set_temperature	temperature_control_node	Allows the user to set the desired temperature via the user interface node.

3.4 Actions

Action	Server Node	Client Node	Description
/dim_lights	light_control_node	user_interface_node	Provides gradual light dimming with real-time progress feedback and status.

4. rqt_graph Analysis



The rqt_graph provides a visual representation of the interaction between nodes, topics, and actions in the Smart Home Control System. Here is a breakdown:

1. Central Node:

The `/user_interface_node` is the central point of interaction, acting as the main interface for the user.

2. Temperature Control:

The `user_interface_node` publishes desired temperature commands to the `/temperature_command` topic.

The `/temperature_control_node` subscribes to this topic, processes the desired temperature, and regulates the simulated thermostat accordingly.

3. Light Control:

The `user_interface_node` sends light commands via the `/light_command` topic to the `/light_control_node`.

Gradual dimming is implemented through the /dim_lights action server in the light_control_node.

Feedback (/dim_lights/_action/feedback) and status (/dim_lights/_action/status) topics are used to update the user_interface_node on the progress of dimming.

4. Security System:

The /security_node publishes the security system state to the /security_state topic.

The user_interface_node subscribes to this topic to provide feedback to the user.

5. Communication Flow:

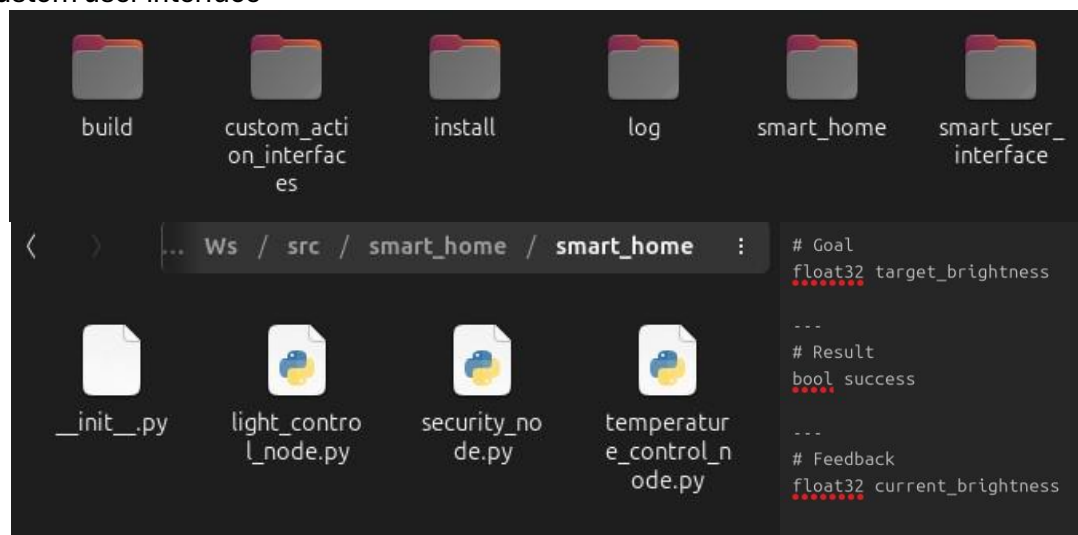
Commands (temperature, lighting, and security) flow from the user_interface_node to their respective nodes.

Feedback and status flow back to the user_interface_node for user visibility.

5. System file and nodes

We can see Below our workspace and the node that we have on the smart home package. Our workspace source code contains three main packages:

- Smart home
- User interface
- Custom user interface



Inside the custom user interface, we have a action file only, because we used the stander SRV and messages that are available in the ROS environment without doing a custom interface for them. Above is our action file and its component.

6. System Flow Summary

The user_interface_node sends:

- Desired temperature commands to the temperature_control_node (via /temperature_command).
- Lighting commands to the light_control_node (via /light_command or /dim_lights action server).
- Security system control signals.

The temperature_control_node:

- Receives commands and regulates the simulated thermostat.
- Provides temperature feedback.

The light_control_node:

- Responds to lighting commands.
- Handles gradual dimming through the /dim_lights action server.

The security_node:

- Publishes updates to /security_state for the user interface.

The user_interface_node:

- Aggregates all feedback and updates for user interaction.

7. Conclusion

The Smart Home Control System provides an integrated and modular design for simulating a smart home environment. The use of ROS2 topics, services, and actions enables smooth communication and flexibility between subsystems. The system successfully achieves:

- Real-time temperature control with user-specified input.
- Lighting control with both direct commands and gradual dimming functionality.
- A simulated security system for monitoring and feedback.

The rqt_graph visualizes the communication flow effectively, highlighting the central role of the user_interface_node in orchestrating all operations. This modular architecture ensures scalability for adding future features or subsystems, making it an ideal foundation for smart home automation projects.